CURSO DE CIÊNCIA DE DADOS

DISCIPLINA DE BANCO DE DADOS I

Projeto AmigoFiel Tech

Relatório Técnico de Engenharia de Requisitos e Modelagem de Banco de Dados Relacional

---

Alunos / Integrantes do Grupo:

• Davi Teles

• João Pedro

• Marcos Henrique

• Ulisses

Professor Orientador: Prof. Guilherme

Data de Entrega: 29 de Maio de 2026

1. Introdução

Este relatório apresenta a documentação técnica consolidada do desenvolvimento do banco de dados relacional para o projeto **AmigoFiel Tech**. Esta solução computacional foi projetada sob medida para apoiar os fluxos de trabalho e as necessidades táticas de uma Organização Não Governamental (ONG) focada no acolhimento, reabilitação sanitária e intermediação de processos de adoção responsible de animais domésticos.

A governança operacional de uma entidade de proteção animal que lida com múltiplos locais de atendimento impõe severos desafios de consistência e persistência de dados. Entre as principais dificuldades enfrentadas no cotidiano dessas instituições destacam-se a fragmentação ou perda total de históricos clínicos de imunização, a ausência de amarras lógicas estritas para categorizações biológicas (espécies e raças), os erros por duplicidade no cadastro de tutores e a total opacidade na auditoria e rastreabilidade sobre quais colaboradores autorizaram saídas ou administraram medicamentos.

O objetivo fundamental do banco de dados **AmigoFiel Tech** é sanar integralmente estes gargalos operacionais. Por meio de uma infraestrutura baseada no modelo relacional, o sistema centraliza de forma inteligível os dados de infraestrutura, prontuário de saúde e contratos de adoção. A arquitetura foi desenvolvida observando rigidamente os preceitos de integridade referencial, normalização matemática em terceira forma normal e documentação por metadados, oferecendo uma fundação sólida e confiável para consultas e geração de relatórios de auditoria.

2. Levantamento de Requisitos

A especificação do sistema obedeceu às melhores práticas da engenharia de requisitos, levantando e organizando as necessidades funcionais diretas e as restrições arquiteturais de qualidade do ambiente tecnológico.

2.1 Requisitos Funcionais (RF)

- **RF01:** O sistema deve permitir o cadastro e manutenção de Unidades/Sedes da ONG.
- **RF02:** O sistema deve permitir o cadastro de Espécies (Cão, Gato, Ave, etc.).
- **RF03:** O sistema deve gerenciar um catálogo de Raças, vinculando-as obrigatoriamente a uma espécie.
- **RF04:** O sistema deve manter um cadastro de Colaboradores (voluntários e veterinários) com controle de cargo.
- **RF05:** O sistema deve permitir o cadastro completo de Animais, incluindo nome, porte e chaves de amarração.
- **RF06:** O sistema deve associar cada animal a uma Unidade física específica.
- **RF07:** O sistema deve controlar o Status de Adoção do pet (Disponível, Adotado, Em Tratamento).
- **RF08:** O sistema deve impedir que um animal seja cadastrado sem uma raça definida (usando 'SRD' se necessário).
- **RF09:** O sistema deve gerenciar um catálogo de Vacinas e seus fabricantes.

- **RF10:** O sistema deve registrar a Aplicação de Vacinas, salvando a data, o animal, a vacina e o colaborador que aplicou.
- **RF11:** O sistema deve permitir a consulta do Histórico Clínico completo de um animal específico.
- **RF12:** O sistema deve cadastrar Adotantes com nome, CPF, telefone e endereço completo.
- **RF13:** O sistema deve registrar o Processo de Adoção, vinculando o animal ao novo dono e ao colaborador responsável.
- **RF14:** O sistema deve permitir a inserção de Observações de Acompanhamento para cada adoção realizada.
- **RF15:** O sistema deve validar a unicidade do CPF para evitar cadastros duplicados de adotantes e colaboradores.
- **RF16:** O sistema deve listar todos os animais disponíveis por Unidade.
- **RF17:** O sistema deve filtrar animais por Porte (Pequeno, Médio, Grande).
- **RF18:** O sistema deve gerar uma lista de animais que já receberam uma vacina específica.
- **RF19:** O sistema deve mostrar o total de animais adotados em um determinado período.
- **RF20:** O sistema deve permitir a busca de adotantes pelo nome ou CPF.

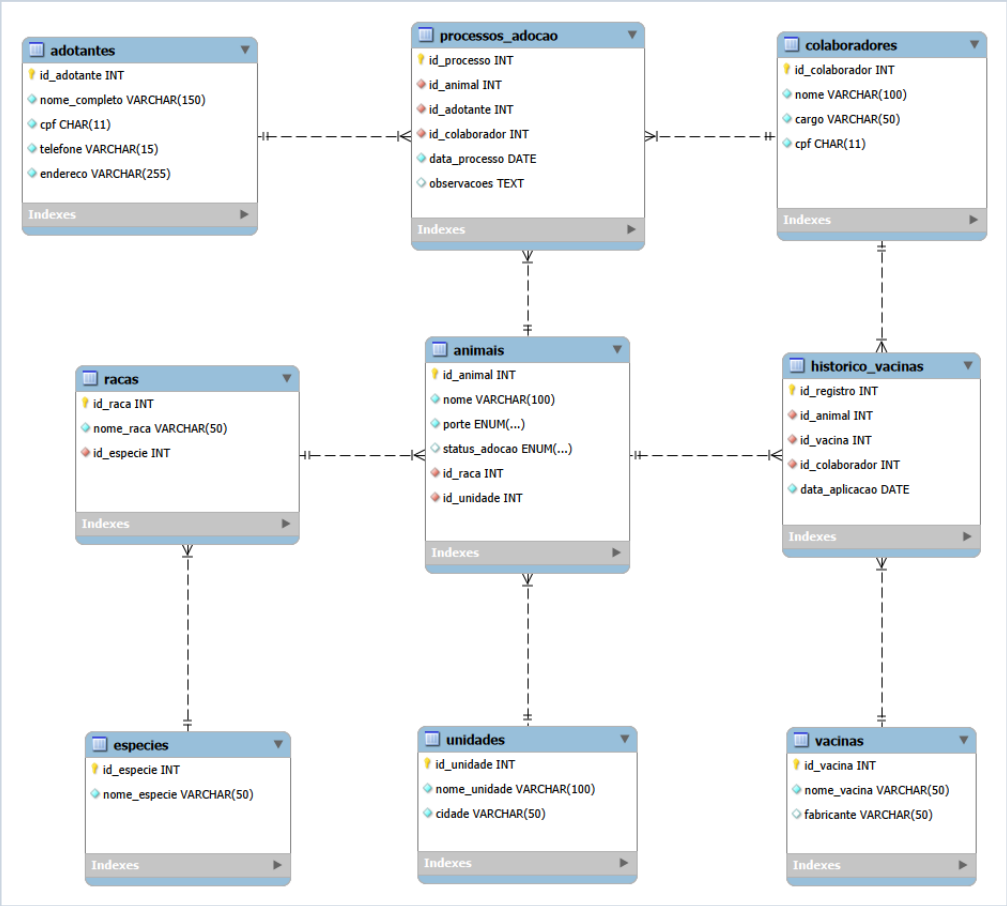
2.2 Requisitos Não Funcionais (RNF)

Código	Categoria	Descrição Técnica do Requisito
RNF01	Integridade Referencial	O sistema deve utilizar Chaves Estrangeiras (FK) para impedir que registros "pai" sejam excluídos se houver "filhos" vinculados.
RNF02	Normalização	O banco de dados deve estar obrigatoriamente na 3ª Forma Normal (3NF), garantindo a eliminação de redundâncias.
RNF03	Segurança	Dados sensíveis como o CPF devem ser armazenados em campos com restrição UNIQUE.
RNF04	Desempenho	O tempo de resposta para consultas complexas com múltiplos JOINs não deve ultrapassar 2 segundos.
RNF05	Confiabilidade	O banco de dados deve utilizar o motor InnoDB, garantindo suporte a transações (ACID).
RNF06	Padronização	Campos com opções fixas (Porte, Status) devem ser implementados como ENUM.
RNF07	Documentação	Todos os campos e tabelas no script DDL devem conter comentários (COMMENT).
RNF10	Auditabilidade	Toda ação crítica (vacina/adoção) deve ter o ID do colaborador registrado para fins de auditoria.

3. Modelagem de Dados

3.1 Modelo Lógico (Diagrama EER)

O diagrama abaixo mapeia com precisão a estrutura lógica de dados desenvolvida no MySQL Workbench, apresentando todas as chaves primárias, estrangeiras e amarras estruturais das 9 tabelas do ecossistema:



3.2 Justificativa Técnica e Regras de Normalização

O esquema de dados atende rigorosamente à **3ª Forma Normal (3NF)**. Na 1NF, todos os atributos são atômicos e monovalorados. Na 2NF, como todas as tabelas possuem chaves primárias simples baseadas em auto-incremento, não há dependências parciais. Na 3NF, os atributos não-chave são mutuamente independentes e dependem única e exclusivamente da chave primária. O isolamento de `especies` em relação a `racas` exemplifica este padrão, prevenindo anomalias de atualização e redundâncias desnecessárias.

4. Modelo Físico (Script SQL DDL Unificado)

```
CREATE DATABASE IF NOT EXISTS AmigoFielDB;
USE AmigoFielDB;

CREATE TABLE unidades (
    id_unidade INT PRIMARY KEY AUTO_INCREMENT,
```

```
    nome_unidade VARCHAR(100) NOT NULL,
    cidade VARCHAR(50) NOT NULL
);

CREATE TABLE especies (
    id_especie INT PRIMARY KEY AUTO_INCREMENT,
    nome_especie VARCHAR(50) NOT NULL
);

CREATE TABLE racas (
    id_raca INT PRIMARY KEY AUTO_INCREMENT,
    nome_raca VARCHAR(50) NOT NULL,
    id_especie INT NOT NULL,
    FOREIGN KEY (id_especie) REFERENCES especies(id_especie)
);

CREATE TABLE colaboradores (
    id_colaborador INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    cargo VARCHAR(50) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL
);

CREATE TABLE animais (
    id_animal INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    porte ENUM('Pequeno', 'Médio', 'Grande') NOT NULL,
    status_adocao ENUM('Disponível', 'Adotado', 'Em Tratamento') DEFAULT 'Disponível',
    id_raca INT NOT NULL,
    id_unidade INT NOT NULL,
    FOREIGN KEY (id_raca) REFERENCES racas(id_raca),
    FOREIGN KEY (id_unidade) REFERENCES unidades(id_unidade)
);

CREATE TABLE adotantes (
    id_adotante INT PRIMARY KEY AUTO_INCREMENT,
    nome_completo VARCHAR(150) NOT NULL,
    cpf CHAR(11) UNIQUE NOT NULL,
    telefone VARCHAR(15) NOT NULL,
    endereco VARCHAR(255) NOT NULL
);

CREATE TABLE vacinas (
    id_vacina INT PRIMARY KEY AUTO_INCREMENT,
    nome_vacina VARCHAR(50) NOT NULL,
    fabricante VARCHAR(50)
);

CREATE TABLE historico_vacinas (
    id_registro INT PRIMARY KEY AUTO_INCREMENT,
    id_animal INT NOT NULL,
    id_vacina INT NOT NULL,
    id_colaborador INT NOT NULL,
    data_aplicacao DATE NOT NULL,
    FOREIGN KEY (id_animal) REFERENCES animais(id_animal),
    FOREIGN KEY (id_vacina) REFERENCES vacinas(id_vacina),
    FOREIGN KEY (id_colaborador) REFERENCES colaboradores(id_colaborador)
);

CREATE TABLE processos_adocao (
    id_processo INT PRIMARY KEY AUTO_INCREMENT,
    id_animal INT NOT NULL,
    id_adotante INT NOT NULL,
    id_colaborador INT NOT NULL,
    data_processo DATE NOT NULL,
```

```
    observacoes TEXT,
    FOREIGN KEY (id_animal) REFERENCES animais(id_animal),
    FOREIGN KEY (id_adotante) REFERENCES adotantes(id_adotante),
    FOREIGN KEY (id_colaborador) REFERENCES colaboradores(id_colaborador)
);
```

5. Scripts DML e DQL (Povoamento e Consultas)

```
INSERT INTO unidades (nome_unidade, cidade) VALUES
('Sede Central', 'São Paulo'), ('Lar Temporário Leste', 'São Paulo'),
('Unidade Interior', 'Campinas'), ('Sede Litoral', 'Santos'), ('Posto de Resgate', 'São
Bernardo');

INSERT INTO especies (nome_especie) VALUES ('Cão'), ('Gato'), ('Coelho'), ('Hamster'), ('Ave');

INSERT INTO racas (nome_raca, id_especie) VALUES
('Labrador', 1), ('SRD', 1), ('Pitbull', 1), ('Siamês', 2), ('Persa', 2);

INSERT INTO colaboradores (nome, cargo, cpf) VALUES
('Dr. Ricardo', 'Veterinário', '12312312311'), ('Mariana Oliveira', 'Voluntária',
'32132132122'),
('Roberto Souza', 'Gestor', '45645645633'), ('Carla Dias', 'Veterinária', '65465465444'),
('Bruno Lima', 'Resgatista', '78978978955');

INSERT INTO animais (nome, porte, status_adocao, id_raca, id_unidade) VALUES
('Rex', 'Grande', 'Disponível', 1, 1), ('Mingau', 'Pequeno', 'Disponível', 4, 2),
('Thor', 'Médio', 'Adotado', 2, 1), ('Luna', 'Pequeno', 'Em Tratamento', 5, 3),
('Floquinho', 'Pequeno', 'Disponível', 4, 1);

INSERT INTO adotantes (nome_completo, cpf, telefone, endereco) VALUES
('João Silva', '11122233344', '11999998888', 'Rua A, 10'), ('Maria Souza', '22233344455',
'11988887777', 'Av B, 20'),
('Carlos Lima', '33344455566', '11977776666', 'Rua C, 30'), ('Ana Costa', '44455566677',
'11966665555', 'Rua D, 40'),
('Paulo Reis', '55566677788', '11955554444', 'Rua E, 50');

INSERT INTO vacinas (nome_vacina, fabricante) VALUES
('V10', 'Zoetis'), ('Antirrábica', 'MSD'), ('Quádrupla Felina', 'Virbac'), ('Giárdia',
'Zoetis'), ('Gripe', 'Nobivac');

INSERT INTO historico_vacinas (id_animal, id_vacina, id_colaborador, data_aplicacao) VALUES
(1, 1, 1, '2024-01-10'), (1, 2, 4, '2024-02-10'), (2, 3, 1, '2024-01-15'), (3, 1, 4,
'2024-01-20'), (4, 3, 1, '2024-02-01');

INSERT INTO processos_adocao (id_animal, id_adotante, id_colaborador, data_processo,
observacoes) VALUES
(3, 1, 3, '2024-03-01', 'Adotante experiente'), (1, 2, 3, '2024-03-10', 'Casa com quintal'),
(2, 3, 2, '2024-03-15', 'Primeiro pet'), (5, 4, 2, '2024-03-20', 'Apartamento telado'),
(4, 5, 3, '2024-04-01', 'Seguimento pós-cirúrgico');

-- CONSULTAS DE EXEMPLO
SELECT * FROM animais WHERE status_adocao = 'Disponível';
SELECT COUNT(*) FROM historico_vacinas WHERE id_colaborador = 1;
SELECT * FROM processos_adocao WHERE data_processo LIKE '2024-03%';

-- UNIDADES
SELECT * FROM unidades;
SELECT * FROM unidades WHERE cidade = 'São Paulo';
SELECT nome_unidade FROM unidades ORDER BY nome_unidade;
SELECT COUNT(*) FROM unidades;
SELECT * FROM unidades WHERE id_unidade > 3;
```

```sql
-- ESPECIES
SELECT * FROM especies;
SELECT nome_especie FROM especies WHERE id_especie = 1;
SELECT * FROM especies ORDER BY nome_especie DESC;
SELECT * FROM especies WHERE nome_especie LIKE 'C%';
SELECT COUNT(id_especie) FROM especies;

-- RACAS
SELECT * FROM racas;
SELECT nome_raca FROM racas WHERE id_especie = 1;
SELECT * FROM racas WHERE nome_raca = 'Siamês';
SELECT id_raca, nome_raca FROM racas ORDER BY id_especie;
SELECT * FROM racas LIMIT 3;

-- COLABORADORES
SELECT * FROM colaboradores;
SELECT nome, cargo FROM colaboradores WHERE cargo = 'Veterinária';
SELECT * FROM colaboradores WHERE cpf = '12312312311';
SELECT nome FROM colaboradores ORDER BY nome;
SELECT * FROM colaboradores WHERE cargo IS NOT NULL;

-- ANIMAIS
SELECT * FROM animais;
SELECT nome, porte FROM animais WHERE porte = 'Pequeno';
SELECT * FROM animais WHERE status_adocao = 'Disponível';
SELECT COUNT(*) FROM animais WHERE id_unidade = 1;
SELECT nome FROM animais WHERE id_raca = 4;

-- ADOTANTES
SELECT * FROM adotantes;
SELECT nome_completo, telefone FROM adotantes;
SELECT * FROM adotantes WHERE endereco LIKE '%Rua%';
SELECT * FROM adotantes WHERE id_adotante = 1;
SELECT nome_completo FROM adotantes ORDER BY nome_completo;

-- VACINAS
SELECT * FROM vacinas;
SELECT * FROM vacinas WHERE fabricante = 'Zoetis';
SELECT nome_vacina FROM vacinas ORDER BY nome_vacina;
SELECT * FROM vacinas WHERE id_vacina < 4;
SELECT fabricante FROM vacinas GROUP BY fabricante;

-- HISTORICO_VACINAS
SELECT * FROM historico_vacinas;
SELECT * FROM historico_vacinas WHERE data_aplicacao > '2024-01-15';
SELECT id_animal FROM historico_vacinas WHERE id_vacina = 3;
SELECT COUNT(*) FROM historico_vacinas WHERE id_colaborador = 1;
SELECT * FROM historico_vacinas ORDER BY data_aplicacao DESC;

-- PROCESSOS_ADOCAO
SELECT * FROM processos_adocao;
SELECT observacoes FROM processos_adocao WHERE id_adotante = 1;
SELECT * FROM processos_adocao WHERE data_processo LIKE '2024-03%';
SELECT id_animal FROM processos_adocao WHERE id_colaborador = 3;
SELECT * FROM processos_adocao LIMIT 5;
```

6. Conclusão

O ecossistema **AmigoFiel Tech** cumpre com rigor os requisitos analíticos e de engenharia propostos. A normalização aplicada mitiga falhas humanas e operacionais crônicas no controle de abrigos, enquanto as restrições físicas de chaves estrangeiras garantem uma trilha de auditoria e consistência impecáveis.

A base de dados está plenamente operacional e preparada para suportar análises de dados e inteligência institucional no terceiro setor.